Agentic AI for Business and FinTech (FTEC5660)
Individual Project Due date: 3/1, midnight
Name: YANG Jinyue Student ID:1155242966

Research Report on Agentic System Project
AI-Trader: Can AI Beat the Market?
Original GitHub Repository: [https://github.com/HKUDS/AI-Trader]

Summary

This project evaluates an LLM-based quantitative trading agent (AI-Trader) powered by Gemini-2.5-Flash. The agent uses a microservices architecture (FastMCP) and integrates five independent tools: Math computation, Alpha Vantage news retrieval, Trading tools, Local_price data, and Cryptocurrency trading tools. The primary goal is to replicate the benchmark agent's capabilities, including autonomously retrieving historical stock prices (e.g., NVDA), obtaining market sentiment through external news APIs, executing logical buy/sell decisions, and accurately tracking portfolio value during the simulated backtesting period.

1. Setup Notes (Env, Data, Keys, Compute)

Scope: Due to the limit of API rate limits, costs, and compute time, this reproduction experiment is designed to select a single, high-volatility asset (NVDA) to rebuild it based on Gemini-2.5-flash. Focusing on a single ticker allowed for an in-depth analysis of the agent's core reasoning logic, state management, and computation accuracy without exhausting API budgets.

Environment: Windows Subsystem for Linux (WSL - Ubuntu), Python 3.12.
(A pure Linux venv was strictly required to prevent data conflicts).

Data: Local files for historical stock prices (data/daily_prices_NVDA.) acting as the offline database for the LocalPrices tool.

Model: Gemini-2.5-flash, Base URL(Default Google Generative AI endpoint), temperature=0.0, max_tokens=2048.

Keys: GEMINI_API_KEY for the core reasoning engine.
ALPHAVANTAGE_API_KEY for the Search tool to fetch market news.

Tools: Model Context Protocol (MCP) was used to bind get_price and web_search.

Compute: Local execution for the Python orchestration and FastMCP servers.
Heavy reasoning and NLP tasks offloaded to the Gemini API.

LLM: Gemini 2.5 flash

Experimental Design (3-Run):
Considering the non-determinism of Large Language Models (LLMs), a single test execution is insufficient. To ensure the reliability, both the baseline agent and the modified agent were subjected to 3 independent runs across the exact same date range (2025-10-01 to 2025-11-07).

2. Target & Metric

2.1 Core Objectives

This project aims to validate the core hypothesis of the original repository: that agents based on large language models (LLMs)can construct and optimize portfolios through multi-step tool use. The replication focuses on evaluating the agent's logical reasoning capabilities in complex financial environments, the accuracy of toolchain calls, and the contribution of real-time market information to decision quality.

2.2 Task Definition

To achieve verification within the limit of resources and API quotas, this project selects one miniaturized experimental task.

- Asset：NVDA, a highly volatile asset, was chosen as the test subject.
- Time: October 1 to November 7, 2025.
- Technical logic: By focusing on a single asset, this experiment enables a more detailed observation of the agent's state transitions and decision boundaries when responding to specific market events.

In addition to standard replication, this project designed and conducted an independent study on Prompt Optimization change by adding specific constraints.

Metric:

Performance Metrics Across Markets:

(CR)Cumulative Return

(SR)Sortino Ratio

(Vol)Volatility

(MDD)Maximum Drawdown.

Mark-to-Market Total Portfolio Value at the end of the simulation.

The ground-truth NVDA closing price on Nov 7, 2025 ($180.50) was used.

Initial capital was $10,000.

3. Baseline Results & Comparison

3.1 Baseline Result

Because the trading choices of 15+ assets are limited to a single asset (NVDA), the baseline results are not directly comparable to the original gross return. However, they demonstrate the agent's functional baseline.

Baseline	Final Portfolio Value	CR	MDD	Vol	SR
Baseline 1	$9,965.68	-0.34%	8.97%	39.80%	0.17
Baseline 2	$9,967.00	-0.33%	8.66%	35.13%	0.13
Baseline 3	$9,904.32	-0.96%	9.49%	35.00%	-0.12
Mean	$9,945.67	-0.54%	9.04%	36.64%	0.06

Table 1: result of Baseline Group

3.2 Comparison with Original Repository

The following table shows the results from Baseline (NVDA Single-Asset) against the reference metric from the original study.

Metric	Baseline	Reference	Delta (Δ)
Cumulative Return (CR)	-0.54%	-0.06%	-0.48%
Max Drawdown (MDD)	9.04%	7.73%	+1.31%
Annualized Volatility (Vol)	36.64%	22.06%	+14.58%
Sortino Ratio (SR)	0.06	0.09	-0.03

Table 2: Comparison between Baseline Group and Reference

3.3 Analysis

Liquidity & Allocation

The logs reveal a critical engineering bottleneck: the initial capital allocation phase. On 2025-10-03, the Agent executed a near-total liquidation of USD to acquire 52 shares of NVDA, leaving a residual cash balance of only $176.16(Baseline 2).

Engineering Implication: Because the stock price ($180+) consistently exceeded the remaining cash, the Agent was mathematically locked out of further "Buy" actions.

Behavioral Note: This represents a "Greedy Initialization" flaw. Without a programmed Position Sizing module, the LLM treats the experiment as a binary "All-in or Hold" scenario, effectively neutralizing the "Multi-step" advantage of the MCP architecture.

Sentiment Saturation: The news tools consistently returned positive long-term outlooks for NVDA. In the absence of a "Sell" signal from the sentiment analysis, and the inability to "Buy" due to cash limits, the LLM defaulted to the only logical state remaining: Stasis.

Latency and Connectivity Robustness

API Timeouts: Several news retrieval attempts (e.g., Oct 27) failed because of rate limit on the AlphaVantage endpoint.

System Resilience: The base_agent.py logic handled these None returns. Instead of crashing, the system defaulted to a "Hold" state based on internal state memory rather than real-time telemetry. This proves robustness of the error-handling wrappers implemented in the provided framework.

Volatility of Baseline (36.64%) and Reference (22.06%)

Technical Cause: This divergence is likely due to Asset Concentration. The reference data likely utilized a diversified portfolio(15+), whereas this replication focused only on NVDA.

Logic Verification: The Agent correctly identified the bullish AI trend (referencing xAI and news in the logs) but failed to implement a Stop-Loss. While the "Reasoning" layer is working, the "Execution" lacks a quantitative trigger.

4. Modification

4.1 Modification Result

According the results from the baseline group, system prompt is updated to optimize the performance.

1. API FALLBACK: If the News tool fails or returns empty, you MUST switch to Technical Analysis immediately. Do NOT default to "Hold".

2. MANDATORY TOOL: You are REQUIRED to call `tool_trade.py` in every single step to verify trends.

3. LIQUIDITY: If Cash < $200, you MUST actively look for SELL signals (e.g., High RSI) to free up cash. Do not just sit on a full position.

Experiment	Final Portfolio Value	CR	MDD	Vol	SR
Trial 1	$10,240.97	+2.41%	6.29%	31.09%	1.07
Trial 2	$10,229.66	+2.30%	7.28%	28.53%	1.51
Trial 3	$10,345.93	+3.46%	5.16%	25.42%	1.74
Mean	**$10,272.19**	**+2.72%**	**6.24%**	**28.35%**	**1.44**

Table 3: result of Experimental Group

4.2 Comparison

To evaluate the impact of the prompt engineering strategies, comparative analysis is conducted between Baseline (default configuration) and Experimental Group (updated prompts). Each group has 3 independent trials for the stochastic nature of the LLM's decision-making process.

The table summarizes the key performance metrics across all six trials.

Metric	Baseline (Mean)	Experimental (Mean)	Delta
Final Portfolio Value	$9,945.67	$10,272.19	+$326.52
Cumulative Return (CR)	-0.54%	+2.72%	+3.26%
Max Drawdown (MDD)	9.04%	6.24%	-2.80%
Annualized Volatility(Vol)	36.64%	28.35%	-8.29%
Sortino Ratio(SR)	0.06	1.44	+1.38

Table 4: Comparison between Baseline and Experimental Group

Return Reversal: The optimized prompt successfully reversed the negative expectancy of the baseline strategy, improving the mean cumulative return from -0.54% to +2.72%.

Risk Mitigation: The introduction of liquidity constraints significantly reduced downside risk. The Maximum Drawdown (MDD) decreased by approximately 2.80% (from 9.04% to 6.24%), indicating better capital preservation during market corrections.

Risk-Adjusted Performance: The Sortino Ratio, which penalizes only downside volatility, improved drastically from 0.06 to 1.44. This suggests that the volatility observed in the

Experimental Group was largely due to upside moves rather than harmful drawdown.

4.2 Engineering Analysis

Through a deeper inspection of the transaction logs, I find that it shows differences in agent behavior, which explain the differences in quantitative results.

Baseline Group:

• Action: Immediate allocation of >90% of capital into a single asset (NVDA) within the first 2-3 trading days.

• Inaction: Once fully invested, the agent ceased trading activities (Action: None or Hold), regardless of market signals or API availability issues.

• Result: The portfolio became a proxy for the underlying asset's volatility (beta ≈ 1), with no mechanism to hedge against the November pullback. This exposes a hidden assumption in the default system: the reliance on a bull market.

Experimental Group: dynamic decision-making capabilities by the new prompt:

• Cash Buffer: By introducing the LIQUIDITY instruction (which triggers sell signals when cash is below $200), the agents holds a cash position about 5-15% during high-volatility periods, adhering to the LIQUIDITY instruction.

•API Fallback: Introduces a FALLBACK system, switching to technical analysis when news tools fail to execute. This prevents the baseline group from being stuck in a 'passive holding' state due to API rate limits, thereby enhancing the system's risk resilience.

•Mandatory Tool Enforcement: Requires the tool_trade.py script to validate trends at each step via the command, resolving the 'trading stagnation' issue in the baseline group. The agent executed partial sell orders during the late-October, effectively locking in profits.

• **Diversification (Trial 3): Trial-3 agent spontaneously made a pair trade by selling NVDA to purchase AAPL. This behavior was not explicitly hard-coded but emerged from the instruction to "minimize risk," proving that the LLM can derive financial concepts from risk directives.**

The variance in the Experimental Group (Returns: +2.41%, +2.30%, +3.46%) shows the non-deterministic nature of generative agents and a direct result of the enhanced prompt architecture.

5. Debug Diary

Environment & Connectivity (The "Proxy" Hurdle)

Issue: Initial execution failed during the Gemini API handshake. The WSL2 environment was unable to route traffic to Google's servers, resulting in ConnectTimeoutErrors.

Resolution: Investigated the network bridge between Windows and WSL2. Implemented a global proxy export in .bashrc using the host IP (cat /etc/resolv.conf).

Cross-Platform venv Corruption:

Issue: Moving the project folder from Windows to WSL caused pip to throw massive invalid metadata entry 'name' warnings due to incompatible OS binaries.

Resolution: Executed a hard reset by deleting the Windows environment (rm -rf venv) and

natively rebuilding the virtual environment inside Linux.

Microservice Port Conflicts (Ghost Processes):
Issue: MCP services failed to start (5/5 failed) with Address already in use errors on ports 8000-8005 after a manual interrupt (Ctrl+C).

Resolution: Used OS-level process management (pkill -9 -f python) to forcefully terminate lingering detached background services, freeing the ports.

WSL DNS Failure (External API Crash):
Issue: The Search service crashed with Temporary failure in name resolution when hitting Alpha Vantage. WSL failed to inherit the Windows host's DNS settings.

Resolution: Overrode the Linux network configuration by manually injecting global DNS (echo "nameserver 8.8.8.8" > /etc/resolv.conf), successfully restoring external API connectivity.

Version Control ("FastMCP" Conflict)
Issue: The experiment crashed upon initializing tools due to a version mismatch of fastmcp. Specifically, the latest FastMCP class signature in version 2.0 differed from the code provided in the standard @mcp.tool().
Resolution: Transitioned to a decoupled terminal architecture. I manually established a dedicated service terminal for the tool layer: Executed python3 agent_tools/tool_crypto_trade.py in a separate terminal session.

Data-Time Misalignment:
Issue: Agent reported Data not found for date and refused to trade.

Resolution: Identified that the default_config.json backtest dates exceeded the local data. Realigned the config dates to perfectly match the historical data scope.

Tool Data Type Mismatch:
Issue:*tool_get_price_local.py* failed to return data because the LLM generated ISO date strings that did not match the local format.

Resolution: Added a Normalizationlayer using *datetime.fromisoformat()* to normalize date inputs before querying the data.

State Retrieval Crash:
Issue: *tool_trade.py* crashed with a KeyError: 'CASH' when reading *positions.json* if a specific ticker wasn't initialized.

Resolution: Replaced direct dictionary indexing with the *.get()* method (e.g., p*ositions.get('CASH', 0)*) to ensure the agent loop would not terminate unexpectedly.

Multimodal Response Type Conflict
Issue: On Nov 06, the system crashed with TypeError: sequence item 0: expected str instance, list

found. The original code *"\n".join([msg.content for msg in tool_msgs])* was too rigid to handle these high-dimensional responses.

Resolution: Injected a Type-Casting Defense Mechanism into *agent/base_agent/base_agent.py* to flatten responses: *tool_response = "\n".join([str(msg.content) for msg in tool_msgs])*

6. Conclusion

6.1 Reproducibility Analysis

This experiment achieved 100% reliable reproducibility in engineering architecture. Several tests verifies the stability of the following modules:

- MCP protocol docking: The communication link between LLM and external tools (Math, Search, LocalPrices) shows exceptional stability after environmental restoration.
- State machine consistency: Throughout 28 consecutive trading days, the read/write operations in positions.jsonl maintained state integrity without any loss, while the mathematical calculations for capital and positions remained strictly accurate.
- Data pipeline: The system reliably parses complex JSON nested structures from multiple APIs, demonstrating the robustness of its underlying data cleansing logic.

6.2 Not Reproducible

While the underlying logic is reproducible, the specific transaction trajectories show inherent biases that cannot be eliminated:

- Decision timing and quantity—the exact number of shares to sell (6 or 5) on October 22 may vary slightly across different experimental batches.
- Deviation in reasoning: Even with Temperature set to 0, the LLM may still select different tool invocation sequences during a 30-step ReAct loop due to minor fluctuations in Token probabilities.
- External API dependencies: Due to call quotas and network in Alpha Vantage and Web Search, news summaries retrieved by the same prompt may vary slightly across different times, causing drift in the Agent's observation context.

6.3 Cause

Information drift: The online search tool returns real-time internet data. Even for historical backtesting, the search interface updates its content over time, causing the Agent to see inconsistent reference information each time.

Stochasticity in Agent Loops: Multi-step Planning has cumulative effects. Minor variations in the initial reasoning phase (e.g., changing the response format from String to List) are amplified through feedback loops, ultimately leading to divergent trading behaviors.